

Fundamentos de la programación 1

Grado en Desarrollo de Videojuegos

Examen de convocatoria extraordinaria. Curso 24/25

Indicaciones generales:

- Se entregará únicamente el archivo `Program.cs` con el programa pedido.
 - Las líneas 1 y 2 serán comentarios de la forma:
// Nombre y apellidos
// Laboratorio, puesto
 - **Lee atentamente el enunciado** e implementa el programa tal como se pide, con los métodos, parámetros y requisitos que se especifican. No puede modificarse la representación propuesta, ni alterar los parámetros de los métodos especificados. No se especifica el modo de paso de los parámetros en los métodos (`out`, `ref`, ...), que debe determinar el alumno. Pueden implementarse todos los métodos adicionales que se consideren oportunos, especificando claramente su cometido, parámetros, etc.
 - **El programa debe compilar y funcionar correctamente, y estar bien estructurado y comentado.** Se valorarán la claridad, la concisión y la eficiencia.
 - La **entrega**: se realizará a través del servidor FTP de los laboratorios.
Importante: comprobar el archivo entregado en el puesto del profesor.
-

Vamos a implementar un *rompecabezas de piezas deslizantes*. Consta de una cuadrícula rellena con piezas etiquetadas con letras y una posición vacía, que permite el deslizamiento de las piezas adyacentes, en horizontal o en vertical. El objetivo es recolocar las piezas en orden ascendente, de izquierda a derecha y de arriba a abajo, dejando la posición vacía en la esquina inferior derecha. Por ejemplo, para tamaño 4×5 puede tener este aspecto:

b	c	m	o	r
i	f	n	p	e
a		s	j	g
h	q	d	l	k

a	b	c	d	e
f	g	h	i	j
k	l	m	n	o
p	q	r	s	

A la izquierda se muestra un estado no resuelto del juego, con el cursor en la posición (1,3) y el hueco vacío en la posición (2,1). En esta situación hay 4 posibles movimientos: 'f' hacia abajo, 'a' hacia la derecha, la 's' a la izquierda o 'q' hacia arriba. El tablero de la derecha corresponde al estado resuelto del juego, con las letras ordenadas y el hueco abajo a la derecha.

Vamos a implementar nuestra versión del juego para **tamaño genérico** $N \times M$, con $N, M \geq$

3. Se proporciona una plantilla en el archivo `Program.cs`.

```
class Program{
    static Random rnd = new Random(); // generador de números aleatorios
    struct Coord { // coordenadas para representar posiciones
        public int fil, col; }
    static void Main() {
        char[,] tab; // tablero de juego
        Coord pos; // posición actual del cursor
```

La matriz `tab` contiene las letras del juego. El tipo `Coor` permite representar coordenadas `fil, col` en el tablero ((0,0) corresponde a la esquina superior izquierda). La variable `pos`, de tipo `Coor`, guarda la posición actual del cursor en el tablero, que servirá para realizar los deslizamientos.

Implementaremos los siguientes métodos:

- [1 pt] `void Inicializa(int fils, int cols, char [,] tab, Coor pos)`: crea el tablero `tab` de tamaño `fils×cols`, inicializa las casillas con los caracteres de la 'a' en adelante y ' ' en la última posición (i.e., en estado resuelto), e inicializa `pos` en (0,0).
Recordemos que puede utilizarse el operador `++` con el tipo `char`. Por ejemplo, si `c` es una variable con el valor 'e', la expresión `c++` es correcta y se evalúa a 'f'. Por otro lado, para evitar caracteres que no corresponden a letras, asumimos que el número de casillas del tablero es menor o igual que 27 (de la 'a' a la 'z' hay 26 caracteres en ASCII, que con el blanco ' ', da un máximo de 27 caracteres para el juego).
- [1 pt] `void Desordena(char [,] tab)`: reordena aleatoriamente las letras de `tab`. Para ello **utilizaremos el siguiente algoritmo**: intercambia cada una de las casillas con otra determinada aleatoriamente. El generador de números aleatorios `rnd` ya está definido en la plantilla y cada llamada `rnd.Next(a,b)` da un entero aleatorio del intervalo `[a,b)`.
- [1 pt] `void Render(char[,] tab, Coor pos)`: muestra el tablero `tab` y sitúa el cursor en pantalla, en la posición `pos`. El formato será el mostrado en los ejemplos (con los caracteres separados por blanco). Para situar el cursor en pantalla se utilizará el método `Console.SetCursorPosition(left,top)`.
- [0.5 pt] `void ProcesaInput(char [,] tab, Coor pos, char c)`: mueve la posición del cursor `pos` en función de `c`: para los valores 'u', 'd', 'l', 'r' mueve el cursor hacia arriba, abajo, izquierda, derecha, respectivamente. Si la posición obtenida se saliese de los límites del tablero, el método no hace nada.

En la plantilla se proporciona implementado el método `LeeInput`, que devuelve 'u', 'd', 'l', 'r' para las teclas de dirección (up, down, left, right). Con estos métodos se puede implementar una primera versión del bucle principal, que muestre el tablero y permita mover el cursor. A continuación implementaremos el deslizamiento, que se hará pulsando la barra espaciadora sobre el carácter a deslizar (`LeeInput` devuelve 's' para la barra espaciadora).

- [1 pt] `Coor LocalizaHueco(char[,] tab)`: busca la posición del tablero donde se encuentra el hueco en blanco y la devuelve en un valor de tipo `Coor`.
- [1.5 pt] `void Desliza(char [,] tab, Coor pos)`: si el hueco en blanco es adyacente a la posición `pos`, desliza la letra a dicho hueco. A continuación se muestra un ejemplo de deslizamiento. A la izquierda, se muestra el cursor sobre la letra 'g'. Al deslizarla se obtiene el tablero de la derecha.

c	o	h	m	n
k	e	a	i	d
b	j		g	l
q	f	r	p	s

c	o	h	m	n
k	e	a	i	d
b	j	g		l
q	f	r	p	s

Para implementar este método, en primer lugar se obtiene la posición del hueco utilizando `LocalizaHueco`. Si las posiciones del cursor y el hueco son adyacentes se desliza la letra al hueco, es decir, se intercambia el contenido de ambas posiciones. Si el hueco y el cursor no son adyacentes, el método no hace nada.

Con este método se extenderá `ProcesaInput` para hacer el deslizamiento con `'s'` y ya se puede probar el funcionamiento básico del juego. Para completarlo implementaremos los siguientes métodos:

- [1 pt] `bool Terminado()`: determina si el juego está terminado, es decir, si las piezas están ordenadas por filas y columnas, con el hueco al final, tal como se muestra al principio del enunciado (imagen de la derecha).
- [1 pt] `void Lee(string file, char[,] tab, Coord pos)`: abre el archivo indicado en `file` y crea e inicializa `tab` de acuerdo al contenido de ese archivo. Además inicializa el cursor `pos` en (0,0). El archivo de entrada tendrá dos líneas al principio indicando el número de filas y columnas, y a continuación la matriz de caracteres, como en el siguiente ejemplo (que se proporciona en el archivo de ejemplo `"letras"`):

```
3
4
g c b f
i e   d
j h k a
```

Ahora completaremos el método `Main` con el siguiente comportamiento:

- [2 pt] `void Main()`: da a elegir al usuario las opciones de leer un tablero de archivo o generar uno nuevo aleatorio (de tamaño 4×5). A continuación arranca el juego con ese tablero, permitiendo realizar deslizamientos. Terminará cuando las piezas estén completamente ordenadas (juego terminado) o bien el usuario aborta el juego con `Escape` (`LeeInput` devuelve `'q'`).